

Qt + Snap7 窗口程序 —— 小白上手指南（项目 2）

本项目和项目 1 的区别

	项目 1	项目 2
界面	黑色控制台	图形窗口
用户操作	只能看打印结果	输入 IP、点按钮、看到状态变化
用到的库	snap7	snap7 + Qt6 Widgets
新增概念	—	信号/槽、多线程、事件循环

你需要什么

除了项目 1 的工具外，还需要 Qt6 库。只要装过 Qt 6.x（带 MinGW），这些都在 C:\Qt\6.11.0\mingw_64\ 下面。

项目文件一览

stage2_qt_connect/	
├── CMakeLists.txt	← 构建配置（比项目 1 多了 Qt6 相关设置）
├── main.cpp	← 程序入口（启动 Qt 应用 + 显示主窗口）
├── mainwindow.h	← 主窗口声明（控件、信号、槽）
├── mainwindow.cpp	← 主窗口实现（界面搭建 + 按钮逻辑）
├── plc_worker.h	← 工作线程声明（封装 TS7Client）
├── plc_worker.cpp	← 工作线程实现（真正的连接/断开操作）
├── snap7.h	← snap7 头文件
├── snap7.cpp	← snap7 C++ 封装层
├── snap7.dll	← snap7 运行时库
└── libsnap7_mingw.a	← MinGW 导入库（从 DLL 生成）

核心概念一：信号和槽（Signal & Slot）

这是 Qt 最核心的机制。传统编程中，按钮点击要调用某个函数，你得写回调、函数指针。Qt 把它简化成一句话：

```
连接(谁发出信号, 什么信号, 谁接收, 干什么)
connect(sender, SIGNAL, receiver, SLOT)
```

举个具体的例子：

```
// 当 connectBtn 被点击时，调用 MainWindow 的 onConnectClicked 函数
connect(m_connectBtn, &QPushButton::clicked,
```

```
this, &MainWindow::onConnectClicked);
```

类比理解：

Qt 概念	生活类比
信号 (signal)	门铃响了
槽 (slot)	你去开门
connect()	把门铃电线接到"开门"这个动作上

核心概念二：为什么需要两个线程

`client.ConnectTo(ip, 0, 1)` 是一个**阻塞**操作。如果 PLC 没开机，它会等好几秒才返回"连接失败"。如果这个调用发生在 UI 线程里，整个界面会冻结——窗口拖不动、按钮点不了，直到超时结束。

所以架构如下：



一条完整的"连接"数据流：

1. 用户点"连接"按钮
2. QPushButton 发出 clicked 信号
3. MainWindow::onConnectClicked 收到 → 设置状态为"正在连接..."
4. MainWindow 发出 requestConnect 信号
5. Qt 自动把信号投递到 Worker 线程
6. PlcWorker::connectToPlc 在 Worker 线程执行
7. `m_client->ConnectTo(IP, 0, 1)` ← 可能阻塞几秒
8. 成功 → worker 发出 connected 信号
9. 失败 → worker 发出 errorOccurred 信号

- 10. Qt 自动把信号投递回 UI 线程
- 11. MainWindow 收到信号 → 更新界面（绿色已连接 / 红色错误）

所有跨线程的信号传递，Qt 在内部自动处理。你只需要写好 connect() 就行。

各文件源码讲解

main.cpp (程序入口)

```
#include <QApplication>
#include "mainwindow.h"

int main(int argc, char *argv[])
{
    QApplication app(argc, argv); // 创建 Qt 应用对象（必须有）
    MainWindow w;                  // 创建主窗口
    w.show();                      // 显示窗口
    return app.exec();             // 进入事件循环，等待用户操作
}
```

app.exec() 是 Qt 的“事件循环”——程序不会像控制台一样从上到下跑完就退出，而是一直运行，等待用户点击按钮、敲键盘等操作。

plc_worker.h (Worker 声明)

```
class PlcWorker : public QObject
{
    Q_OBJECT // 必须有这个宏，才能使用信号/槽

public slots: // slot = 被调用的那一端
    void connectToPlc(const QString &ip, int rack, int slot);
    void disconnectFromPlc();

signals: // signal = 发出通知的那一端
    void connected();
    void disconnected();
    void errorOccurred(int code, const QString &message);

private:
    TS7Client *m_client; // 项目 1 里用的那个类
};
```

plc_worker.cpp (Worker 实现)

```

void PlcWorker::connectToPlc(const QString &ip, int rack, int slot)
{
    QByteArray ipBytes = ip.toUtf8();    // QString → const char*
    int result = m_client->ConnectTo(ipBytes.constData(), rack, slot);

    if (result == 0) {
        emit connected();                // 发信号：连上了
    } else {
        QString errMsg = QString::fromUtf8(CliErrorText(result).c_str());
        emit errorOccurred(result, errMsg); // 发信号：出错了
    }
}

```

mainwindow.h (主窗口声明)

```

class MainWindow : public QMainWindow
{
    Q_OBJECT

signals:
    // 发给 Worker 的信号（跨线程）
    void requestConnect(const QString &ip, int rack, int slot);
    void requestDisconnect();

private slots:
    void onConnectClicked(); // 按钮点击
    void onConnected();      // Worker 说连上了
    void onError(int code, const QString &message); // Worker 说出错了

private:
    void setupUi(); // 搭建界面控件
    QThread *m_thread;
    PlcWorker *m_worker;
};

```

mainwindow.cpp 关键部分

① setupUi()——搭界面

```

void MainWindow::setupUi()
{
    setWindowTitle("S7-1200 连接测试");

    m_ipEdit = new QLineEdit("192.168.0.1"); // 输入框
    m_connectBtn = new QPushButton("连接 PLC"); // 按钮
}

```

```

m_statusLabel = new QLabel("未连接");           // 状态文字
m_disconnectBtn = new QPushButton("断开");

// 用布局管理器排列控件 (VBox = 竖向排列, HBox = 横向排列)
// ...

// 连接按钮信号到槽
connect(m_connectBtn, &QPushButton::clicked,
        this, &MainWindow::onConnectClicked);
}

```

② 构造函数——创建 Worker 线程

```

MainWindow::MainWindow(QWidget *parent)
{
    setupUi();

    m_thread = new QThread(this);
    m_worker = new PlcWorker();           // 无 parent, 因为要被移走
    m_worker->moveToThread(m_thread);     // 把 Worker "搬家"到子线程

    // 跨线程信号连接
    connect(this, &MainWindow::requestConnect,
            m_worker, &PlcWorker::connectToPlc); // UI → Worker
    connect(m_worker, &PlcWorker::connected,
            this, &MainWindow::onConnected);    // Worker → UI

    m_thread->start(); // 启动子线程
}

```

`moveToThread(m_thread)` 是关键——之后 `PlcWorker` 的所有槽函数都会在子线程中执行, 不会阻塞 UI。

编译步骤

1. 配置

```

cmake -B build -G "Ninja" \
    -DCMAKE_CXX_COMPILER=g++ \
    -DCMAKE_PREFIX_PATH=C:/Qt/6.11.0/mingw_64

```

比项目 1 多了 `-DCMAKE_PREFIX_PATH=...`, 告诉 CMake "Qt6 装在这个目录"。

2. 编译

```
cmake --build build
```

3. 部署 (补齐 Qt 运行时 DLL)

```
export PATH="/c/Qt/6.11.0/mingw_64/bin:$PATH"
cd build
windeployqt Stage2_QtConnect.exe
```

windeployqt 会自动分析 exe 需要哪些 Qt DLL，把它们连同平台插件一起复制到 build 目录。

4. 运行

部署完成后，直接双击 build\Stage2_QtConnect.exe 即可打开窗口。

CMakeLists.txt 详解

```
cmake_minimum_required(VERSION 3.16)
project(Stage2_QtConnect LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_AUTOMOC ON)           # ← 新增！自动处理 Q_OBJECT 宏

find_package(Qt6 REQUIRED COMPONENTS Widgets) # ← 新增！找 Qt6

add_executable(${PROJECT_NAME}
    main.cpp
    mainwindow.cpp # ← 新增
    plc_worker.cpp # ← 新增
    snap7.cpp
)

target_link_options(${PROJECT_NAME} PRIVATE -static)
target_link_libraries(${PROJECT_NAME} PRIVATE
    Qt6::Widgets # ← 新增！链接 Qt 窗口库
    ${CMAKE_SOURCE_DIR}/libsnap7_mingw.a
    ws2_32
)
```

新增配置	作用
CMAKE_AUTOMOC ON	Qt 的 MOC 预处理器——扫描 .h 中的 Q_OBJECT 宏，自动生成元对象代码
find_package(Qt6 ...)	在系统中查找 Qt6 安装位置

Qt6::Widgets	链接 Qt Widgets 库 (提供窗口、按钮、标签等控件)
--------------	---------------------------------

常见问题

Q: cmake 报 `Could not find a package configuration file provided by "Qt6"`

原因: CMake 不知道 Qt6 装在哪。**解决:** 加 `-DCMAKE_PREFIX_PATH=C:/Qt/6.11.0/mingw_64` (改成你电脑上 Qt 的实际路径)。

Q: 双击 exe 没反应 / 报"找不到 Qt6Core.dll"

原因: exe 旁边没有 Qt 运行时 DLL。**解决:** 运行 `windeployqt Stage2_QtConnect.exe`, 它会自动把需要的文件全拷过来。

Q: 点连接按钮后界面卡死

原因: `snap7` 的 `ConnectTo` 在 UI 线程执行, 阻塞了事件循环。**解决:** 检查 `moveToThread` 是否写了, `m_worker` 是否真的在子线程里。

Q: 按钮点了没反应

原因: 信号和槽没有 connect。**解决:** 检查 `connect(...)` 语句的四个参数, `sender/signal/receiver/slot` 都要配对。

术语速查

术语	一句话解释
QApplication	Qt 应用对象, 管理事件循环 (每个 Qt 程序必须有一个)
事件循环	程序不退出, 不断等待、分发用户的鼠标键盘操作
信号 (signal)	"某件事发生了"的通知
槽 (slot)	收到通知后执行的函数
QThread	Qt 的线程类
moveToThread	把 QObject 移到指定线程, 它的槽函数会在那个线程执行
MOC	Meta-Object Compiler, Qt 的代码预处理器
windeployqt	Qt 发布工具, 自动复制 exe 需要的所有 DLL